Pharmacy Point of Sale System

Biztonsági Sérülékenységek Elemzése

SZTE Információbiztonság kurzus — Web - B esszé

Felhasznált AI prompt dokumentáció

A feladat elemzéséhez az alábbi promptot használtam ChatGPT segítségével.[1]

> Az alábbi PHP/SQLite webalkalmazás (https://github.com/CaTwoPlus/patika) forráskódját elemezd és találd meg az alábbi sérülékenységeket. Segíts a BurpSuite programmal való lekérési paraméterek megjelenítésében. Fő feladatrészek: 1. egy CSRF példa megtalálása: az alkalmazás melyik részén található? Mivel kapcsolatos? Hogy lehetne javítani a kódban? Hogyan lehetne kihasználni és mire? Ehhez a feladatrészhez különösen javaslom a BurpSuite nevű program Proxy funkcionalitását használni, mivel a requestek során nem feltétlenül jelennek meg az URL sávban a GET paraméterek sem. 2. XSS példák detektálása: találjatok legalább két XSS input bevitelére lehetőséget. Az alkalmazás melyik részén található? Hol futnak le a beinjektált kódok? Mit injektáltatok be, hogyan? Proof of conceptként kerüljenek a beadandóba képernyőképek. Ha sehogy sem találtok két injektálásra lehetőséget, akkor vizsgáljátok meg a kódban, hogy mi lehet ennek az oka. Hogy lehet a sérülékenységeket javítani? 3. Két SQLinjection sérülékenység megtalálása. Az egyik egy egyszerűbb (login-al kapcsolatos), a másik viszont összetettebb. Az alkalmazás melyik részén található? Hogyan lehetne kijavítani a kódot? Proof of conceptként kerüljenek a beadandóba képernyőképek. 4. Két egyéb biztonsági/karbantarthatósági kockázat az alkalmazással kapcsolatban. Ez lehet bármivel kapcsolatos (bejelentkeztetés, adatok tárolásának biztonsága stb), csak legyen konkrét.

Az AI által generált elemzést, megállapításokat a segédanyag, illetve internetes keresések alapján ellenőriztem és szerkesztettem meg. A javításokkal kiegészített fork itt érhető el: https://github.com/CaTwoPlus/patika

1. CSRF — Cross-Site Request Forgery

Az AI legegyértelműbb CSRF-jelöltként az account-módosítást és több, egyéb admin műveletet azonosított be a Actions.php végpontjain. Az alap minta a következő.

---

[1] https://chatgpt.com/share/69e23eb0-9aa8-83eb-aea4-18d9ce152c24

A frontend AJAX-ból POST-olja például az `update_credentials` hívást a `./Actions.php?a=update_credentials` URL-re, de sem a `manage_account.php` kérésében,

```
48          $.ajax({
49              url:'./Actions.php?a=update_credentials',
50              method:'POST',
51              data:$(this).serialize(), // nincs CSRF token
```

sem a szerveroldali `Actions.php-ben` nem látszik CSRF-token ellenőrzés (nincs ellenőrző metódus).

```
Actions.php
1   <?php
2   session_start();
3   require_once('DBConnection.php');
4
5   Class Actions extends DBConnection{
6       function __construct(){ ···
8       }
9       function __destruct(){ ···
11      }
12      function login(){ ···
28      }
29      function logout(){ ···
32      }
33      function save_user(){ ···
81      }
82      function delete_user(){ ···
95      }
96      function update_credentials(){ ···
130     }
131     function save_category(){ ···
176     }
177     function delete_category(){ ···
190     }
191     function save_supplier(){ ···
236     }
237     function delete_supplier(){ ···
250     }
251     function save_product(){ ···
296     }
297     function delete_product(){ ···
310     }
311     function save_stock(){ ···
351     }
352     function delete_stock(){ ···
365     }
366     function save_transaction(){ ···
433     }
434     function delete_transaction(){ ···
447     }
448 }
449 $a = isset($_GET['a']) ?$_GET['a'] : '';
450 $action = new Actions();
451 switch($a){ ···
506 }
```

Ugyanez a minta van a save_user, save_category, save_supplier, delete_user és több más állapotváltoztató műveletnél is.

Hogyan lehet kihasználni?

A szerver bízik a bejelentkezett felhasználóban, és egy külső oldal ezt a bizalmat használja ki. Mivel az alkalmazásban nincs token, ha egy admin be van jelentkezve, és megnyit egy rosszindulatú oldalt, az oldal egy automatikusan elküldött

POST-kéréssel kérheti például ezeket: jogosulatlan adatváltoztatás, user-törlés, jelszócsere, készlet/szállító/kategória módosítás.

Javítás

A CSRF sérülékenység javítására a szervernek minden állapotváltoztató kérésnél anti-CSRF tokent kell használnia. A szerver kriptográfiailag biztonságos véletlengenerátorral létrehoz egy tokent, eltárolja azt a felhasználó szerveroldali sessionjében, majd a kliensnek elküldi például hidden form mezőben vagy AJAX kéréshez használt változóként. A kliens minden érzékeny POST kérésben visszaküldi ezt a tokent, a szerver pedig ellenőrzi, hogy az megegyezik-e a sessionben tárolt értékkel. A token legyen kellően hosszú, ne legyen kitalálható, és legyen sessionhöz vagy időkorláthoz kötve.

Fontos, hogy állapotváltoztató műveleteket ne GET, hanem POST típusú kérésekkel végezzünk. Kiegészítő védelemként a szerver ellenőrizheti az Origin és Referer HTTP fejléceket is. Különösen érzékeny műveletek előtt további felhasználói megerősítés is kérhető, például jelszó újra megadása. Emellett az XSS sérülékenységeket is meg kell szüntetni, mert azok lehetővé tehetik a CSRF token megszerzését és a védelem megkerülését.

Konkrét javítási javaslat

- `index.php`-ban generáljuk a tokent:

```php
index.php
<?php
session_start();
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
```

- Például a `manage_account.php`-ben hozzáadjuk a tokent:

```php
<h3>Manage Account</h3>
<hr>
<div class="col-md-6">
    <form action="" id="user-form">
        <input type="hidden" name="id" value="<?php echo isset($user_id) ? $user_id : '' ?>">
        <input type="hidden" name="csrf_token" value="<?php echo htmlspecialchars($_SESSION['csrf_token'], ENT_QUOTES, 'UTF-8'); ?>">
```

```php
$.ajax({
    url:'./Actions.php?a=update_credentials',
    method:'POST',
    data:$(this).serialize(), // hidden mező már tartalmazza CSRF tokent
    dataType:'JSON',
```

- És az `Actions.php`-ben ellenőrizzük a token meglétét:

```php
    function verify_csrf() {
        if (
            !isset($_POST['csrf_token']) ||
            !isset($_SESSION['csrf_token']) ||
            !hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'])
        ) {
            echo json_encode([
                'status' => 'failed',
                'msg' => 'Invalid CSRF token'
            ]);
            exit;
        }
    }
    function update_credentials(){
        verify_csrf();
```

2. XSS — Cross-Site Scripting

Az első, nagyon erős példa a **kategória leírása**. A `manage_category.php` a `name` és `description` mezőket escape nélkül tölti vissza az űrlapba.

```
15            <input type="text" name="name" autofocus id="name" required
16            class="form-control form-control-sm rounded-0" value="<?php echo isset($name) ? $name : '' ?>">
```

```
19        <textarea name="description" id="description" cols="30" rows="3"
20        class="form-control rounded-0" required><?php echo isset($description) ? $description : '' ?></textarea>
```

A `view_category.php` ugyanígy nyersen írja ki a `name` és `description` értékeket HTML-be. Ez klasszikus stored XSS.

```
18            <div class="fs-6"><b>Category Name:</b></div>
19            <div class="fs-5 ps-4"><?php echo isset($name) ? $name : '' ?></div>
20        </div>
21        <div class="w-100 mb-1">
22            <div class="fs-6"><b>Description:</b></div>
23            <div class="fs-6 ps-4"><?php echo isset($description) ? $description : '' ?></div>
24        </div>
```

A második jó példa a **felhasználó teljes neve / username**. A `manage_user.php` escape nélkül tölti vissza a `fullname` és `username` mezőket,

```
13        <div class="form-group">
14            <label for="fullname" class="control-label">Full Name</label>
15            <input type="text" name="fullname" id="fullname" required class="form-control
16            form-control-sm rounded-0" value="<?php echo isset($fullname) ? $fullname : '' ?>">
17        </div>
18        <div class="form-group">
19            <label for="username" class="control-label">Username</label>
20            <input type="text" name="username" id="username" required class="form-control
21            form-control-sm rounded-0" value="<?php echo isset($username) ? $username : '' ?>">
22        </div>
```

a `users.php` pedig nyersen írja ki a `fullname`-ot és a `username`-ot a táblázatba,

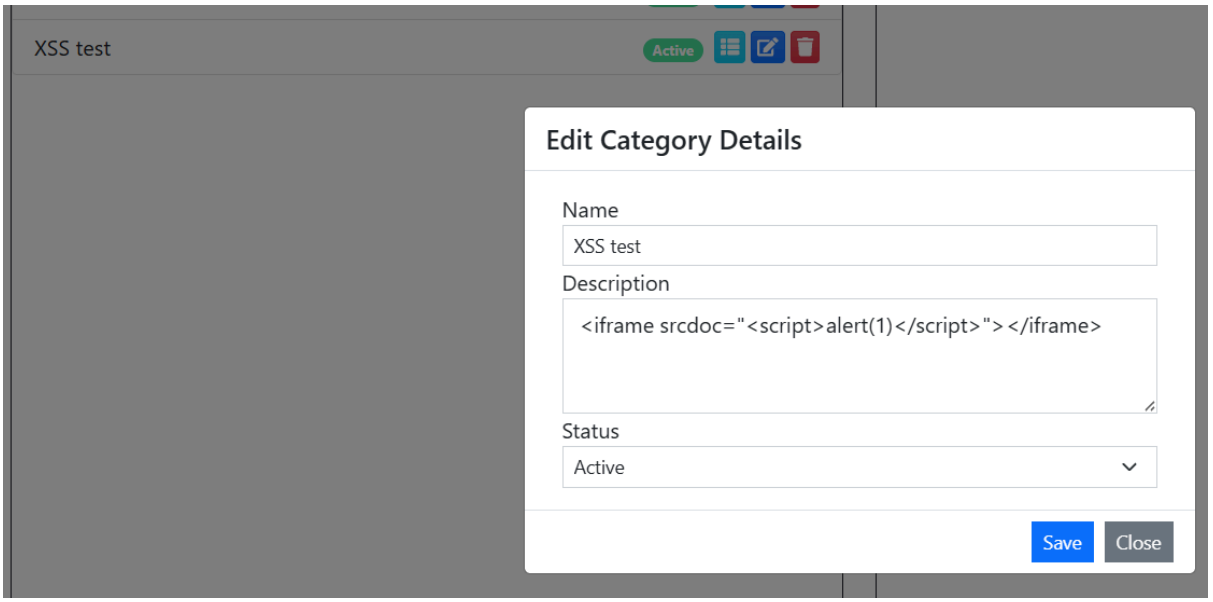
```
36                <td class="py-0 px-1"><?php echo $row['fullname'] ?></td>
37                <td class="py-0 px-1"><?php echo $row['username'] ?></td>
```

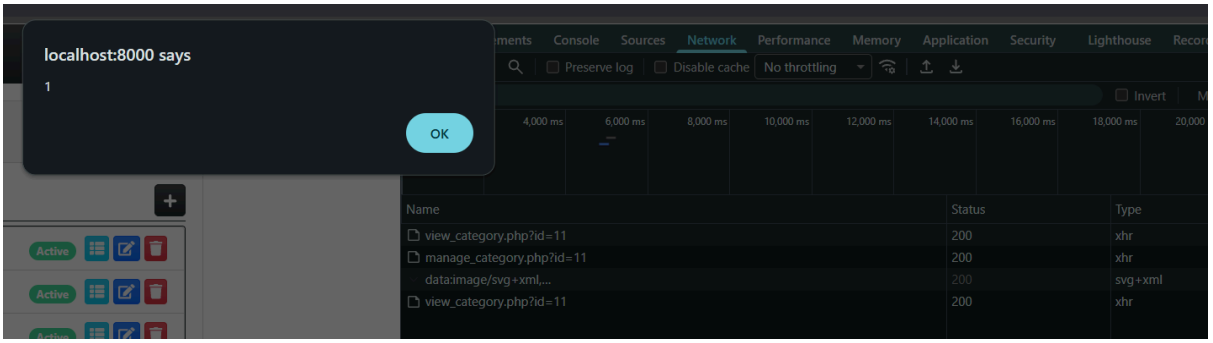
sőt a `fullname` bekerül egy HTML attribútumba is (`data-name`).

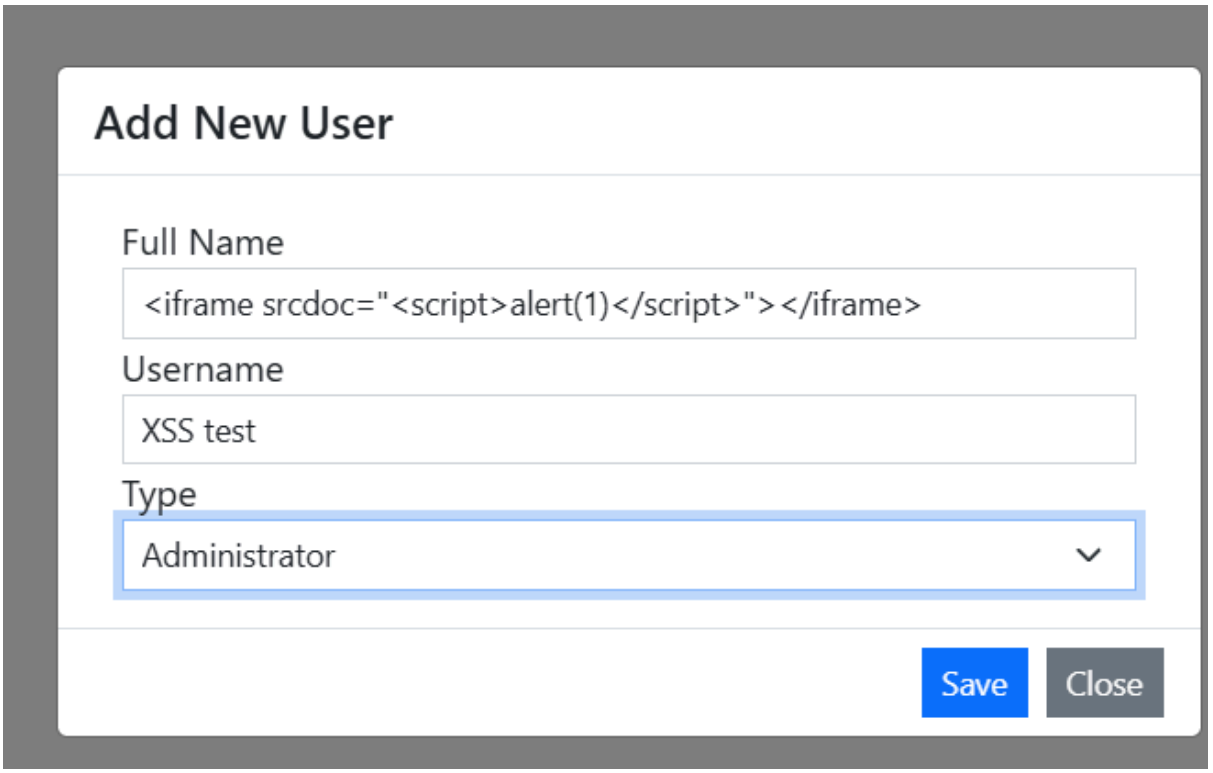
```
46                <li><a class="dropdown-item delete_data" data-id = '<?php echo $row['user_id'] ?>'
47                data-name = '<?php echo $row['fullname'] ?>' href="javascript:void(0)">Delete</a></li>
```

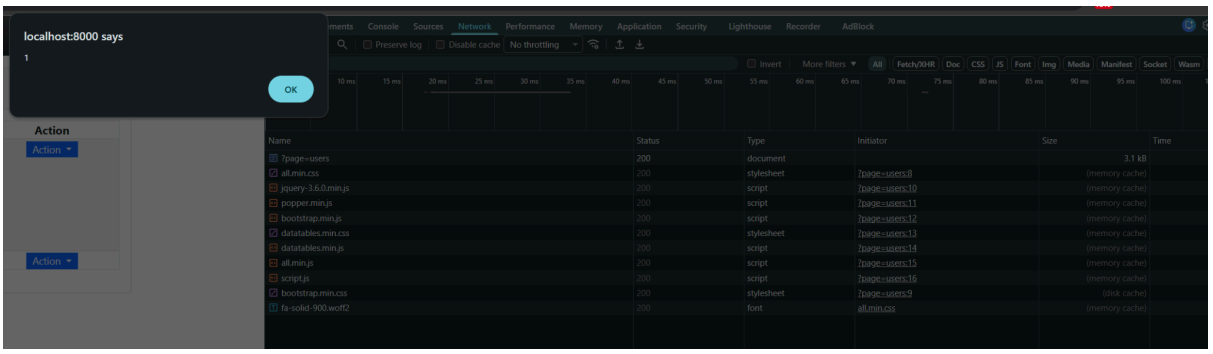
Ez szintén stored XSS.

XSS injektálások









Így tehát az értéket / payloadot a böngésző HTML-ként értelmezi, majd lefuttatja a scriptet. Ezzel akár session hijacking is lehetővé válik, amikor is a ‘<script>alert(document.cookie)</script>’ értékkel a document.cookie tartalma elküldhető egy külső szerverre, megszerezve az admin session-sütijét.

XSS injektálások javítása

Az XSS sérülékenységek elsődleges oka, hogy az alkalmazás a felhasználótól vagy adatbázisból származó adatokat HTML-escape nélkül jeleníti meg. Emiatt a böngésző a bevitt tageket és eseménykezelőket valódi HTML/JavaScript tartalomként értelmezi. A javítás során minden ilyen kimeneti ponton alkalmazni kell a megfelelő escape-elést, PHP esetén a htmlspecialchars($ertek, ENT_QUOTES, 'UTF-8') függvényt. Ez biztosítja, hogy a <, >, ", ' és egyéb speciális karakterek ne markupként, hanem szövegként jelenjenek meg.

Fontos, hogy ne csak a beküldött adatokat ellenőrizzük, hanem a megjelenítéskor is escape-eljünk, mert az adatbázisból visszaolvasott tartalom sem tekinthető automatikusan megbízhatónak. Az escape-elést mindig a megjelenítési kontextushoz kell igazítani: HTML-ben és attribútumokban htmlspecialchars, JavaScript kontextusban például json_encode használata javasolt.

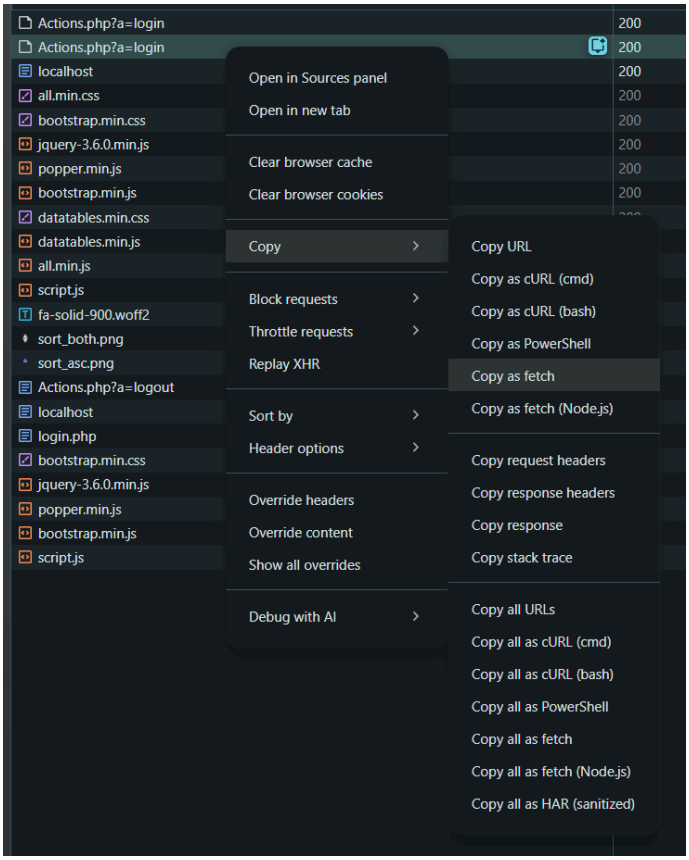
Kiegészítő védelemként érdemes HttpOnly sütiket használni, hogy a kliensoldali JavaScript ne férhessen hozzá közvetlenül a session sütikhez. Továbbá Content Security Policy alkalmazásával csökkenteni lehet az inline szkriptek futásának lehetőségét, mivel ezzel definiálhatók hogy mely erőforrásokat töltheti be az oldal.

Konkrét javítási javaslatra példa

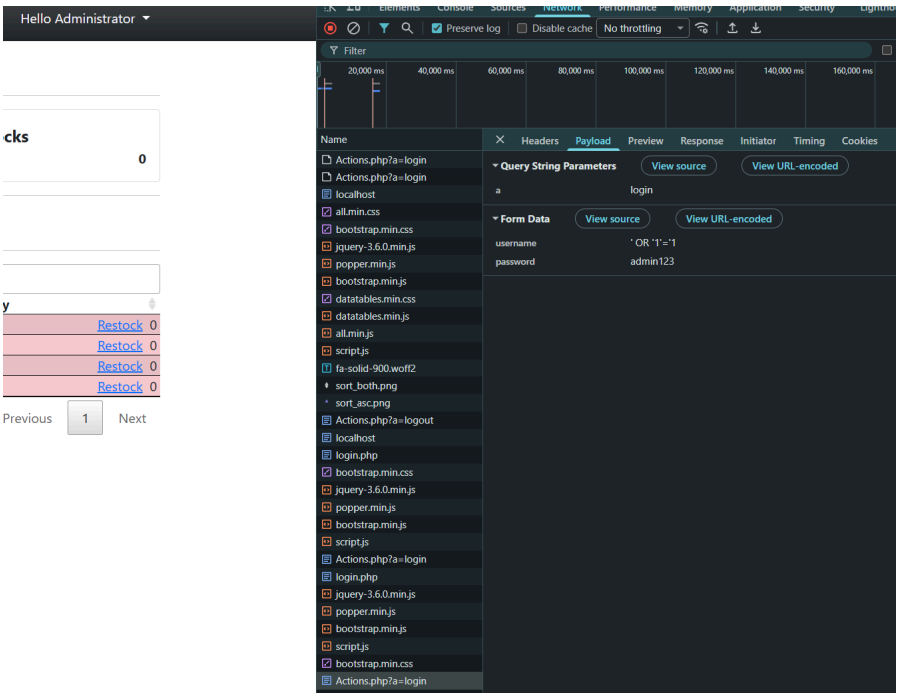
```
15            <input type="text" name="fullname" id="fullname" required class="form-control
16            form-control-sm rounded-0" value="<?php echo htmlspecialchars($row['fullname'], ENT_QUOTES, 'UTF-8'); ?>">
17        </div>
18        <div class="form-group">
19            <label for="username" class="control-label">Username</label>
20            <input type="text" name="username" id="username" required class="form-control
21            form-control-sm rounded-0" value="<?php echo htmlspecialchars($row['username'], ENT_QUOTES, 'UTF-8'); ?>">
22        </div>
```

3. SQL Injection

Egyszerű SQLi — bejelentkezés (Actions.php, login())



A logikai “' OR '1'='1” (idézőjelek nélkül) kell a username értékeként adni, majd frissíteni az oldalt.



Való igaz, bejutottam, mivel username érték közvetlenül belefűződik a SELECT lekérdezésbe.

```
12    function login(){
13        extract($_POST);
14        $sql = "SELECT * FROM user_list where username = '{$username}' and `password` = '".md5($password)."' ";
```

3b. Összetettebb SQLi

Például a `manage_category.php` elején a kód közvetlenül ezt futtatja:

```
manage_category.php
1   <?php
2   require_once("DBConnection.php");
3   if(isset($_GET['id'])){
4   $qry = $conn->query("SELECT * FROM `category_list` where category_id = '{$_GET['id']}'");
```

Vagyis az `id` query paraméter nyersen kerül az SQL-be. Ugyanez a sérülékenység sok más helyen is előfordul:

```
$_GET['id']                                   Aa ab .*
Replace                                       AB
18 results in 9 files - Open in editor - Search with AI
manage_category.php                           M  2
  if(isset($_GET['id'])){
  ...category_list` where category_id = '{$_GET['id']}'");
manage_product.php                            2
  if(isset($_GET['id'])){
  ...product_list` where product_id = '{$_GET['id']}'");
manage_stock.php                              2
  if(isset($_GET['id'])){
  ...stock_list` where stock_id = '{$_GET['id']}'");
manage_supplier.php                           2
  if(isset($_GET['id'])){
  ...supplier_list` where supplier_id = '{$_GET['id']}'");
manage_user.php                               M  2
  if(isset($_GET['id'])){
  ...user_list` where user_id = '{$_GET['id']}'");
view_category.php                             2
  if(isset($_GET['id'])){
  ...category_list` where category_id = '{$_GET['id']}'");
view_product.php                              2
  if(isset($_GET['id'])){
  ...category_id where p.product_id = '{$_GET['id']}'");
view_receipt.php                              2
  if(isset($_GET['id'])){
  ...` where transaction_id = '{$_GET['id']}'");
view_supplier.php                             2
  if(isset($_GET['id'])){
  ...supplier_list` where supplier_id = '{$_GET['id']}'");
```

Javítás

Az SQL injection sérülékenység elsődleges javítása a prepared statementek és bindolt paraméterek használata. Ennek lényege, hogy a felhasználótól érkező adat

nem az SQL lekérdezés szövegébe kerül be konkatenálással, hanem külön paraméterként kerül átadásra. Így a bemenet nem tudja megváltoztatni a lekérdezés szerkezetét, csak adatként lesz kezelve.

Kiegészítő védelemként fontos az inputvalidáció is. Az egyes mezőknél ellenőrizni kell, hogy az adott input megfelel-e a várt formátumnak, hosszúságnak és karakterkészletnek. Például az azonosítók legyenek numerikusak, a felhasználónevek csak megengedett karaktereket tartalmazzanak, a dátumok pedig megfelelő formátumúak legyenek. Ez azonban nem helyettesíti a prepared statementeket, csak kiegészíti azokat.

Fontos továbbá, hogy ne csak a közvetlen felhasználói inputot tekintsük megbízhatatlannak. Más forrásból – például fájlból, korábban eltárolt adatbázisrekordból vagy más rendszerekből – származó adatok is tartalmazhatnak rosszindulatú vagy hibás tartalmat, ezért ezeket is biztonságosan kell kezelni.

Konkrét javítási javaslatra példák

```php
    function login(){
        extract($_POST);

        $username = $_POST['username'] ?? '';
        $password = $_POST['password'] ?? '';

        $stmt = $this->prepare("SELECT * FROM user_list WHERE username = :username AND password = :password");
        $stmt->bindValue(':username', $username, SQLITE3_TEXT);
        $stmt->bindValue(':password', md5($password), SQLITE3_TEXT);

        $qry = $stmt->execute();
        $data = $qry->fetchArray(SQLITE3_ASSOC);

        if($data){
            foreach($data as $k => $v){
                if(!is_numeric($k) && $k != 'password'){
                    $_SESSION['login_' . $k] = $v;
                }
            }

            $this->query("UPDATE user_list SET last_login = CURRENT_TIMESTAMP WHERE user_id = :id");
            $_SESSION['login_type'] = $data['type'];
            return json_encode(['status' => 'success']);
        } else {
            return json_encode(['status' => 'failed', 'msg' => 'Invalid username or password.']);
        }
    }
```

manage_category.php

```php
<?php
require_once("DBConnection.php");
if(isset($_GET['id']) && ctype_digit($_GET['id'])){
    $id = (int) $_GET['id'];

    $stmt = $conn->prepare("SELECT * FROM category_list WHERE category_id = :id");
    $stmt->bindValue(':id', $id, SQLITE3_INTEGER);

    $qry = $stmt->execute();
    $row = $qry->fetchArray(SQLITE3_ASSOC);

    if($row){
        foreach($row as $k => $v){
            $$k = $v;
        }
    }
}
?>
```

4. Egyéb biztonsági és karbantarthatósági kockázatok

Az egyik nagyon konkrét kockázat a **gyenge jelszókezelés**. A login `md5($password)` alapján ellenőriz az `Actions.php`-ben, viszont az MD5 online adatbázis használatával feltörhető.

```
12      function login(){
13          extract($_POST);
14          $sql = "SELECT * FROM user_list where username = '{$username}' and `password` = '".md5($password)."' ";
```

Új user létrehozásakor pedig a jelszó alapból `md5($username)`, tehát az induló jelszó lényegében a felhasználónév. Ez egyszerre gyenge hash és kiszámítható default password. Továbbá, mivel nincs só, két azonos jelszóhoz azonos hash kerül az adatbázisba, ami tovább könnyíti a feltörést.

```
47              if(empty($id)){
48                  $cols[] = 'password';
49                  $values[] = "'".md5($username)."'";
50              }
```

```
472 $a = isset($_GET['a']) ?$_GET['a'] : '';
473 $action = new Actions();
474 switch($a){
475     case 'login':
476         echo $action->login();
477     break;
478     case 'customer_login':
479         echo $action->customer_login();
480     break;
481     case 'logout':
482         echo $action->logout();
483     break;
484     case 'customer_logout':
485         echo $action->customer_logout();
486     break;
487     case 'save_user':
488         echo $action->save_user();
489     break;
490     case 'delete_user':
491         echo $action->delete_user();
492     break;
493     case 'update_credentials':
494         echo $action->update_credentials();
495     break;
496     case 'save_category':
497         echo $action->save_category();
498     break;
499     case 'delete_category':
500         echo $action->delete_category();
501     break;
502     case 'save_supplier':
503         echo $action->save_supplier();
504     break;
505     case 'delete_supplier':
506         echo $action->delete_supplier();
507     break;
508     case 'save_product':
509         echo $action->save_product();
510     break;
511     case 'delete_product':
512         echo $action->delete_product();
513     break;
514     case 'save_stock':
515         echo $action->save_stock();
516     break;
517     case 'delete_stock':
518         echo $action->delete_stock();
519     break;
520     case 'save_transaction':
521         echo $action->save_transaction();
522     break;
523     case 'delete_transaction':
524         echo $action->delete_transaction();
525     break;
526     default:
527     // default action here
528     break;
529 }
```

A másik komoly gond a hiányos hozzáférés- és jogosultságellenőrzés az `Actions.php` végpontokon. Nincs ellenőrizve, hogy be van-e jelentkezve a user a végpontok hívása előtt, így POST kéréssel módosítható meglévő user, illetve új user is hozzáadható. Az sincs ellenőrizve, hogy van-e megfelelő szerepköre a usernek az action végrehajtásához.

Konkrét javítási javaslatra példa

```php
Class Actions extends DBConnection{
    function __construct(){ ···
    }
    function __destruct(){ ···
    }
    function require_login() {
        if (!isset($_SESSION['user']) || !isset($_SESSION['user']['user_id'])) {
            http_response_code(401);
            exit('unauthorized');
        }
    }
    function require_admin() {
        require_login();
        if (!isset($_SESSION['user']['type']) || $_SESSION['user']['type'] != 1) {
            http_response_code(403);
            exit('forbidden');
        }
    }
```

```php
    case 'update_credentials':
        require_login();
        echo $action->update_credentials();
    break;
```

```php
    case 'delete_supplier':
        require_admin();
        echo $action->delete_supplier();
    break;
```

```php
    default:
        http_response_code(400);
        exit('invalid action');
    break;
```